连续五天登上 GitHub Trending 首页的思考

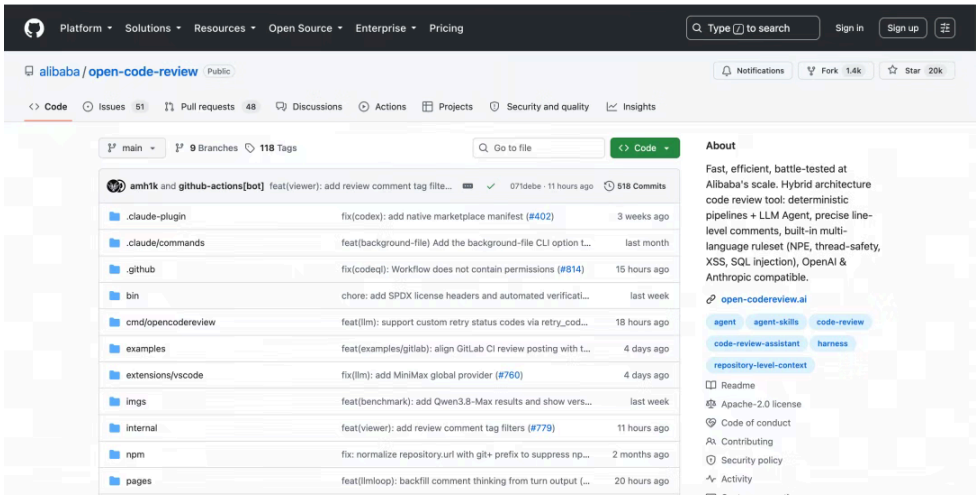
原创 李峥峰 阿里技术 Aug 11, 2026, 3:18 AM 浙江 31 listened



这是2026年的第 49 篇文章

（ 本文阅读时间：约 15 分钟 ）

我们团队在两个月前开源了一个AI Code Review 工具，目前 20k star（项目地址：https://github.com/alibaba/open-code-review）。



100% AI 生成代码、100% AI 评审代码、800 多个 Issue+PR、一百多名外部贡献者、连续 5 天登上 GitHub Trending 首页。

借着这个机会复盘一下数据的背后，我们做对了什么、做错了什么，希望沉淀一些可迁移的方法论给想做开源的开发者，也顺便分享我们极致 AI Coding 的思路。

01
开源之前想清楚核心竞争力和定位

从真实的业务中生长出来，解决真实的问题，而不是为了开源而开源。

我们团队做 AI 代码评审快两年了，在阿里内部有 20k 月活用户，采纳率 30%+、误报率不到 5%、合并到基线的有效建议中近 8 成来自 AI。我们不要求用户处理每条 AI 建议，在这么大的用户基数下这个数据还是比较不错的。说实话一开始没想过做开源， 转折点是从 26 年开始，身边越来越多人跟我说同一个问题：代码是 AI 写的，看不过来，不敢合。这个痛点太真实了，我自己也有。

> Faros AI 发布的《The Acceleration Whiplash》报告也印证了这一点。4,000 个团队、22,000 名开发者两年的遥测数据显示：
>
> AI 编码工具让每个开发者的任务完成度提高了 34%，代码活动激增 210%，但代价是代码重写率飙升 861%、每个 PR 引发的生产事故比率上升 242.7%、PR 平均审查时间延长 441.5%，同时未经审查直接合并的 PR 比例也上升了 31.3%。吞吐量上去了，质量兜不住了。

然后我们去看了市面上的大部分方案，除了头部几个已经商业化的工具，剩下的就是一堆 demo 级的开源项目，AI 时代做个 0-1 的东西太容易了，几天时间就能搞一个。但真正经过大规模验证的开源方案几乎没有。

于是我们看到了一个机会。

我们的定位

我们不宣称自己是最顶尖，但是我们解决的痛点、面向的受众足够广。

1.生产环境验证：不是嘴上说好用，是有 20k 用户生产环境的真实反馈、有 200 个真实 PR 标注的 benchmark 评测集分数。内外同源，我们每个版本内部外部同步发。

2.架构具备特点：“确定性工程 × Agent 协同”的混合架构，不是套个大模型、也不是写个 skill，对代码审查场景中“不能出错”的环节，由工程逻辑而非语言模型来保证，将 AI 的优势集中发挥在它真正擅长的地方 —— 动态决策、动态召回上下文。

3.数据不出本地：只提供框架，不碰用户数据，LLM 你自己选。这点在企业场景里是硬需求。

4.便宜：Token 消耗是 Claude Code + Skills 方案的 1/9。

5.接入方式多：CLI、IDE 插件、各种 Agent 插件、CI-CD、MCP，你想怎么用都行。

6.开源、开放、包容：免费把框架交给社区开发者，大家不用重复造轮子，共同打造出一个好用的工具。

还有一点很反直觉：**主动暴露不足，比营造完美更有效。** 我们在 README 里直接写了很多目前做得还不够好的地方。这样做虽然听起来是给自己减分，但实际效果是：来的人预期对了，用完不会失望，留存反而更高；相反，如果把话说满，用户进来发现不符预期，第一反应就会是“被骗了”，这种负面口碑的传播速度比正面快得多。

后来复盘为什么能引发 100 多个自媒体自发传播，本质上就是这些东西叠在一起：有背书、有数据、有对比、省钱、安全。

02
先完成，再完美

两个关键：

1.窗口期有限。你打磨三个月，别人可能已经占住用户心智了。

2.不完美反而是好事。什么都做好了，外部贡献者没有参与空间，社区起不来。

怎么定义完成？

我们发布的第一个版本只提供了几个内容：

1）基于 Go 语言从零重写的 CLI 工具。
2）一条配置自定义模型的命令，以及兼容 OpenAI、Anthropic 协议。
3）一组评审的命令和一个框架内核。
4）一个配套的 skill，可直接集成到 Claude Code。
5）配套的 Github Action，便于用户直接集成到自己的 Github 仓库。
6）可观测能力，便于用户集成到自己公司内部的系统。

两个月后的现在

从 v1.0.0 到 v1.9.0，100 个正式版本、一百多位贡献者、一百多个 feature commit（其中超过 60% 来自外部 PR）。回头来看，“先完成再完美”不只是一个发布策略，它定义了社区的参与方式 —— 你留出空间，别人就会来填。

内部核心团队搭的是框架骨架：Agent 循环、记忆压缩、Scan 模式、MCP、规则引擎、VSCode 插件、Skill。社区长出来的是血肉：

- **接入方式：**从最初的 CLI + GitHub Action，到 GitLab CI、Gerrit（Jenkins）、Agent Skill、委托模式（复用宿主 Agent 订阅额度）、MCP 客户端。
- **模型生态：**内置 provider 从 3 个扩展到 14 个（含 Ollama 本地、LiteLLM 网关、Eden AI 等），支持 OpenAI、Anthropic、OpenAI Responses 三种协议。
- **语言覆盖：**新增 Python、Rust、Kotlin、C/C++、FreeMarker、GraphQL、Julia、HCL/Terraform、Bicep 等专属评审规则。
- **可观测：**会话查看器（Web UI）、OpenTelemetry 集成优化、W3C traceparent 传播。
- **工程完善：**可恢复会话、token 预算守卫、评论批量分片、Windows 一键安装脚本等。

如果 v1.0.0 那天就把这些全做了再发，至少要多一个月，而且这些能力中有一半是社区用自己的场景「长」出来的需求 —— 我们很难在一开始预见。

这里也必须分享一个教训：我们前期太关注核心功能的完备性了，注意力集中在框架的内核上，在最入口的 LLM 配置方式做得不够简单。结果早期来了一波流量，转化率很低 —— 人来了，用不起来，走了。

后来想明白一件事：易用性就是转化率。「让用户最快能用起来」这件事，本身就属于「先完成」的范畴，不能拖到后面做。因此我们迅速内置了多家主流的模型厂商与 GUI 交互，让用户只需要配置一个 key 就能使用。

03
谨慎增加用户的认知复杂度

这个意识是慢慢长出来的，不是一开始就有。

最典型的例子是 README。随着在社区的发展，越来越多开发者参与进来贡献，开发了各种各样的能力，于是 README 变得越来越大：下载方式（3种）、配置方法（3种）、多种接入方式、高级玩法、生态集成、MCP、Web Viewer、可观测接入等，第一次点进来的人根本不知道该看哪里。

后来我们意识到这个问题，只留：你是谁、为什么选你、怎么快速开始。其他全扔文档站，只留一个标题和跳转链接，将 README 从 1000 行缩短到 200 行。

另一个容易踩坑的地方是 CLI 参数。每加一个参数，用户打`--help`看到的列表就长一行。看起来是“多了个选项”，实际上是多了一层认知负担——用户会想“这个参数我要不要加？不加会怎样？”参数越多，用户越不敢下手。

但这不意味着什么都不能加，关键在于**新增的东西是否让同一个用户面对更多选择**。

举个反例：同时支持 GitLab CI 和 GitHub Actions 集成，这不算增加复杂度。因为用 GitLab 的人根本不会去看 GitHub Actions 的文档，用 GitHub 的人也不会关心 GitLab CI 怎么配。这两类用户不在同一个平面上，互相看不见对方的东西。

我们最近遇到一个问题就是，我们设计了 `--max-tools` 参数用于限制子任务的工具调用轮次，以此来控制极端情况下的工具循环和约束成本。后来社区的开发者又提供了`--max-tool-calls`参数用于控制整个评审的总工具调用次数，以及`--max-tokens-budget` 参数用于物理约束 token 成本。我们拒绝了前者，接受了后者。尽量不要让每个用户陷入该用哪个参数，这才是真正在增加复杂度。

判断标准其实很简单：

```
用户进来 -> 5 分钟理解核心价值并且跑起来 -> 有兴趣再看细节
```

凡是让这条路径变长、变犹豫的改动，都要三思。

04
快速响应：社区活不活就看这个

这是我觉得整个过程中最关键的一个认知。

有个现象很有意思：外部贡献者中最活跃的那批人，几乎都和我们工作时区接近。一开始以为是巧合，后来想明白了 —— 时区接近意味着你提了 PR 我马上能回，正反馈循环快，人就留下来了。反过来说，响应速度本身就是筛选和留存贡献者的机制。

我们的响应节奏大概是这样的：

- 小 bug、小特性：12 小时内发版修复，快的时候 2 小时。
- 社区 Issue & Discussion：提交就能被回复。
- 社区 PR：提交就能被看到，尽快 review & merge。
- 两个月发了 89 个版本，基本上每天一两个。

怎么做到的？靠人肯定不行

坦白说，靠人力根本撑不住这个节奏。背后是一整套 AI Coding 工作流：

内部开发者写的所有代码：100% AI 生成、100% AI 评审。

外部贡献者提交的代码：100% AI 评审。

人干嘛？审查 AI 的输出，做最终决策。

具体来说我们做了这么几个核心 Skill：

`/read-issue`：快速理解 Issue，自动打标签；
`/mk-issue`：基于问题背景创建结构化 Issue ；
`/mkpr`：基于当前改动内容自动创建 PR ；
`/review`：Claude Code + gh cli 评审代码并自动修复；
`/open-code-review`：用 OCR 自身评审代码并自动修复；
`/release-eval`：评估发版改动是否影响核心链路，决定要不要跑评测集（跑一次需要 8 个小时）；
`/tag`：发布新版本；
`/comment`：基于人类的意图润色回复内容，保持友好专业的语气。maintainer 的回复质量直接影响社区氛围，但每条都精心措辞太耗时间了，这个 skill 帮我把“想说的意思”变成“得体的表达”。

工作流的演进也有意思：

- 前期：Claude Code 写代码 -> Skills 评审 -> CC 修复
- 现在：Claude Code 写代码 -> OCR 作为 pre-commit-hook 自动评审 -> CC 自动修复 -> /mkpr 创建评审 -> Github Actions 触发再次评审以及一些围栏任务 -> CC 修复

稳定性靠什么保证？自动化代码评审 + 单元测试 + Lint + CI/CD 流水线 + E2E 评测集(200个PR)等。因为迭代快，所以更需要这些网兜着。

All in Code

这套工作流能跑通，有一个容易被忽略的前提：一切皆代码。

CI/CD 是 YAML，评审规则是 JSON，发版流程是 Makefile + shell，文档站是 MDX，连 Issue 模板和 PR 模板都是 markdown 文件。没有任何关键流程藏在 GUI 后台、wiki 页面或者某个人的脑子里。

这意味着 Agent 可以读、可以改、可以跑。你让 AI 帮你发版，它 cat 一下 Makefile 就知道该执行什么；你让它帮你写评审规则，它`grep`一下现有的 `rule.json` 就知道格式。如果你的发版流程是「点三个按钮、填两个表单、等审批通过」，Agent 根本插不进去。

All in Code 不是什么新理念，DevOps 时代就在喊 Infrastructure as Code。但在 Agent 时代，它的价值被放大了一个数量级——**代码是 Agent 最容易自主操作、也最不容易出错的介质。**你的流程越「代码化」，AI 能接管的比例就越高，你留给自己的就只剩真正需要判断力的决策。

人和 AI 的关系

这两个月下来，我对「人和 AI 怎么协作」这件事有了比较清晰的认知：

AI 适合给你提供多个方案，也适合执行具体实现。但**不要让 AI 自己选方案再执行**，决策权必须在人手里。

这个认知是用一次事故换来的。上 HN 头条前两天，我们让 AI 优化工具调用的逻辑。代码本来就是 AI 写的，我们觉得它应该比我们更熟悉自己写的东西，就没规定怎么改，让它自己决定方案。单测过了，跑了几个例子看着没问题，发了。结果它把一个全局搜索的工具改出了 bug。两天后 HN 的流量涌进来，用户第一次用就踩到了这个坑。你知道这意味着什么——很多人对你的第一印象，就是“这东西不好使”，然后关掉，再也不会回来。痛定思痛，我们定了两条规矩：影响核心链路的改动必须跑完 200 个 PR 的评测集才能发版；AI 写代码时必须给明确的方案约束，不能让它自由发挥。

我的日常时间分配大概是：审查 AI 输出 + 社区互动占 60%，定方向 + 拆 Issue 占 40%。

05
核心开发者搭框架，细节交给社区

一个健康的开源项目需要两种人：稳定的核心贡献者，和源源不断的新人。

稳定贡献者靠什么留？共同荣誉感。大家一起把项目做好，项目越好越有成就感，这是个飞轮。

新人靠什么吸引？Good First Issue。

Good First Issue 的门道

这不是“造”出来糊弄人的任务，是真的需要做、但门槛不高的工作。关键是要写清楚背景、给明确的验收标准、标合理的难度。核心开发者的日常工作里应该持续产出这些 issue——这不是额外工作，是社区建设的一部分。

一个教训

第一次上 GitHub Trending 首页的时候，第二天就没了。复盘下来原因很简单：新人进来没事可做。star 了一下就走了，没有任何后续互动。

第二次上 Trending 的时候，我们做了两件事

1.马上创建一批 good first issue，让新来的人有明确的参与入口。

2.PR 来了就处理，形成“提交就被关注”的体验。

结果连续 5 天 Trending 首页。

逻辑其实很朴素：

```
新人进来 -> 看到能做的事 -> 提了 PR -> 很快被 review & merge
-> 有成就感 -> star / 分享 -> 更多人进来 -> 循环起来了
```

上 Trending 靠的是产品力，但留在 Trending 靠的是社区活跃度。这俩不是一回事。

06
做得易于传播，比自己去传播更重要

我们主动做的传播只有两次，不过得承认，alibaba 本身就是一个巨大的免费流量入口。我们真正可以分享的经验不是“不做传播也能火”，而是有了起始势能之后，怎么让传播自己滚起来。

在一个 AI 驱动创新峰会上分享了一下（OpenCodeReview 是内容的一部分）；公众号有很多自发的宣传之后，我们自己也投稿给了集团旗下的公众号。

后面的事情是自然发生的：

```
峰会分享 -> 社区讨论 -> 公众号自发宣传 -> 上 Trending -> HN 社区有人投稿 ->
```

6 月 6 日上了 HN 头条，star 从 1.5k 直接飙到 4k。

为什么能传播？我事后分析了一下：

- AI Code Review 刚好是当下开发者焦虑的出口：代码越来越多是 AI 写的，怎么保证质量？
- 有品牌背书，可信度够。
- 有 benchmark 数据、有对比图，自媒体可以直接拿去用。
- 痛点真实：省 token、数据安全，这俩是普遍诉求。

本质上，你不需要铺天盖地的营销，只需要吸引更多的潜在传播者以及为你的潜在传播者降低传播门槛。

07
开源能不能成，就看这三层

回过头看，这个项目能走到今天，是三层支撑同时到位了。

第一层：组织信任

开源最大的风险不是技术，是组织。代码公开意味着设计水平全部透明，需要管理层有魄力。更现实的是，开源需要持续投入人力，不被当正式目标支持，纯靠业余时间，节奏撑不住。

两个月 89 个版本，前提是内部把它当正式项目投，不是"有空搞搞"。

第二层：稳定的核心贡献者

社区来来走走正常，但核心团队不能断。新人提了 PR 谁判断该不该合？合了出 regression 谁修？都得靠对项目有深度理解的人。

核心贡献者不是招来的，是从社区里"长"出来的。转化率取决于你的响应速度和认可力度。PR 提了一周没人看，再热情的人也会走。留不住人，不是项目不够好，是反馈不够快。

第三层：真实用户的持续反馈

这层最容易被忽略。开源的是框架，但壁垒是背后用户踩过的坑、验证过的决策。我们开源前已有两年、20k 用户的生产验证，没这些，v1.0.0 就只是又一个 demo。

开源之后，外部用户带来了完全不同的价值：内部环境统一，外部场景千差万别。Gerrit 接入、Ollama 本地模型、Windows 脚本，都是从外部真实场景「长」出来的。

内部用户验证「路走得通」，外部用户发现「还有哪些路要走」；先有用户再有社区，顺序反了会很痛苦。

08
写在最后

现在是做开源最好的时代。

AI 把那些原本吃掉维护者大量时间的重复性工作（Issue 分类、代码评审、测试编写、发版……）自动化了。一个小团队甚至一个人，就能撑起过去十人团队的节奏。门槛降低了，但上限没降——省下来的时间用在真正需要判断力的地方：定方向、做决策、经营社区。

几个我觉得最重要的认知：

别开源一个 demo。 AI 时代做 0–1 太容易了，社区不缺 demo，缺的是经过验证的方案。你的“判决书”越硬（生产数据、benchmark、真实用户规模），别人帮你传播的时候底气就越足。

Trending 不是终点，是起点。 流量来了如果没有承接，就像开了店门但货架是空的。提前准备好 good first issue，不是作弊，是尊重每一个点进来的人的时间。

AI 是 10 倍速的手，但脑子得是自己的。 100% AI 生成代码能成立，前提是人牢牢把住了“做什么”和“做得对不对”。快速响应社区不是因为不睡觉，是因为 AI 工作流把“从 issue 到发版”压缩到了 2 小时。

关键时间线

时间	事件	Star
5 月 21 日	v1.0.0 正式发布	0
5 月 28 日	首次上 Github Trending，第二天掉落	400
6 月 5 日	第二次上 Github Trending	1.5k
6 月 6 日	上 Hacker News 头条	1.5k –> 4k
7 月 23 – 28 日	连续 5 天 Github Trending 首页	10.5k –> 15.5k

***项目官网：**https://open-codereview.ai



欢迎留言一起参与讨论~

阅读 1.0万

留言 9

写留言

Chaos 上海 8月11日 7

很高兴做了点微小的贡献🤏

kite 浙江 8月11日 1

作者赞过



星辰铂金 广东 8月11日

感谢分享！想了解下采纳率 30%+、误报率不到 5%，不采纳又非误报的如何处理?



kite 浙江 8月11日 2

你好，开源版本没有采纳的概念。内部平台上目前是两种: 1. 点击忽略，表明指出的问题正确但是本次不修改。2. 设计了是允许不同仓库自行配置是否必须处理，整体来说会有大概20%左右的评论没有做任何处理(采纳、误报、忽略)。



阿里技术 作者 8月11日

作者在楼上回复啦



magic_pill 北京 8月15日

一直用



开源技术小栈 浙江 8月12日

已经再用啦



欲心21 广东 8月11日

ai 评审两年是？好像也是，之前是chatgpt



碓冰 山东 8月11日

Vibe Coding 的爆发式产能，与核心研发有限的精力，构成了当下最尖锐的矛盾 这个工具出现得很及时 我来学习一下